MODUL PRAKTIKUM 2

Batch Data Ingestion & Processing with Spark

Enterprise Edition (WSL + VS Code + Data Lake + Partitioning)

Programming Language	: **Python**
Development Environment	: ***Linux Server Environment***
Code Editor	: **VS Code** (*Remote* ***WSL***)
Command Line Interface	: **Bash**
Distributed Data Processing Engine	: **PySpark**
Durasi	: **150 menit**

TUJUAN PRAKTIKUM

Mahasiswa mampu:

1. Menggunakan Linux environment (WSL) untuk data engineering
2. Mengintegrasikan VS Code dengan WSL
3. Mengelola virtual environment Python
4. Mengimplementasikan batch data ingestion dengan Spark
5. Menerapkan data cleaning & transformation
6. Mendesain struktur Data Lake (Raw, Clean, Curated)
7. Menggunakan format Parquet
8. Menerapkan partitioning strategy
9. Menghasilkan pipeline enterprise-ready

KONTEKS INDUSTRI

Dalam perusahaan e-commerce, data transaksi harian:

- Volume besar
- Perlu dibersihkan
- Perlu dihitung metrik bisnis
- Disimpan ke data lake
- Siap untuk BI/Analytics

Kita akan mensimulasikan proses tersebut.

🏛 ARSITEKTUR PIPELINE

```
CSV Source
    ↓
Raw Layer
    ↓
Cleaning & Validation
    ↓
Transformation
    ↓
Clean Layer (Parquet)
    ↓
Curated Layer (Aggregation)
    ↓
Partitioned Data Lake
```

Konsep yang digunakan:

- *Batch Processing*
- *Distributed Computing*
- *Medallion Architecture*
- *Columnar Storage*
- *Partition Pruning*

1️⃣ Batch Processing

Batch Processing adalah pemrosesan data dalam jumlah besar sekaligus dalam satu waktu tertentu.

Ciri-ciri:

- Data dikumpulkan dulu
- Diproses secara periodik (harian, mingguan, dll)
- Tidak real-time

Contoh:

- Proses laporan penjualan harian
- Rekap transaksi akhir hari

Dalam praktikum kita:

`Load CSV → Cleaning → Transform → Save`

Itu adalah batch pipeline.

👉 Cocok untuk volume besar yang tidak butuh respon instan.

---

2 Distributed Computing

Distributed Computing adalah teknik membagi pekerjaan komputasi ke banyak core atau mesin agar diproses secara paralel.

Alih-alih satu CPU mengerjakan semuanya:

- Data dibagi menjadi partisi
- Setiap partisi diproses secara bersamaan

Apache Spark bekerja dengan konsep ini.

Keuntungannya:

- Lebih cepat
- Bisa menangani data besar
- Lebih scalable

👉 Ini alasan Spark digunakan untuk big data.

---

3 Medallion Architecture

Medallion Architecture adalah desain data lake berlapis untuk menjaga kualitas data.

Terdiri dari:

- **Raw (Bronze)** → Data mentah
- **Clean (Silver)** → Data sudah dibersihkan
- **Curated (Gold)** → Data siap analitik

Tujuannya:

- Data mentah tidak pernah ditimpa
- Jejak perubahan jelas
- Mudah audit & maintain

Dalam praktikum kita:

data/raw/
data/clean/
data/curated/

👉 Ini standar modern data lake.

---

4 Columnar Storage

Columnar Storage adalah cara menyimpan data berdasarkan kolom, bukan baris.

CSV (row-based):

```
id, name, price
1, A, 100
2, B, 200
Parquet (column-based):
Column id → 1,2
```

```
Column name → A,B
Column price → 100,200
```

Keuntungannya:

- Lebih kecil ukuran file
- Lebih cepat untuk query analitik
- Mendukung compression

👉 Parquet adalah contoh format columnar.

5️⃣ Partition Pruning

Partition Pruning adalah teknik membaca hanya bagian data yang relevan berdasarkan partisi.

Misalnya data dipartisi:

```
category=Electronics/
category=Fashion/
```

Jika query:

```
WHERE category = 'Electronics'
```

Spark hanya membaca folder Electronics.

Hasilnya:

- Lebih cepat
- Lebih hemat resource
- Mengurangi I/O

👉 Ini penting untuk performa big data.

🎯 Ringkasan Singkat

Konsep	Intinya
Batch Processing	Proses data besar sekaligus
Distributed Computing	Proses paralel di banyak core
Medallion Architecture	Data lake berlapis (raw-clean-curated)
Columnar Storage	Simpan data per kolom (efisien analitik)
Partition Pruning	Baca hanya partisi yang dibutuhkan

🧩 BAGIAN A — SETUP WSL (LINUX ENVIRONMENT)

STEP 1 — Install WSL

Buka PowerShell (Run as Administrator):

```
wsl --install
```

Jika belum ada Ubuntu:

```
wsl --install -d Ubuntu
```

Restart jika diminta.

STEP 2 — Jalankan Ubuntu

- Tekan `Windows + S`
- Ketik: `Ubuntu`
- Buat username & password

Jika berhasil muncul:

```
username@DESKTOP:~$
```

🧩 BAGIAN B — INSTALL DEPENDENCY

STEP 3 — Update Linux

```
sudo apt update && sudo apt upgrade -y
```

STEP 4 — Install Java 17

```
sudo apt install openjdk-17-jdk -y
```

Cek:

```
java -version
```

Kenapa?
Spark berjalan di atas JVM.

STEP 5 — Install Python & venv

```
sudo apt install python3-full python3-venv python3-pip -y
```

🧩 BAGIAN C — STRUKTUR PROJECT ENTERPRISE

Masuk ke home:

```
cd ~
mkdir bigdata-project
cd bigdata-project
```

Buat struktur:

```
mkdir -p data/raw
mkdir -p data/clean
mkdir -p data/curated
mkdir logs
mkdir scripts
```

Struktur akhir:

```
bigdata-project/
├── data/
│   ├── raw/
│   ├── clean/
│   └── curated/
├── logs/
├── scripts/
```

🧩 BAGIAN D — INTEGRASI VS CODE DENGAN WSL

STEP 6 — Install VS Code (Windows)

Install dari:
`https://code.visualstudio.com`

STEP 7 — Install Extension WSL

Di VS Code:

- Buka Extensions
- Install: **WSL (Microsoft)**

STEP 8 — Buka Project di Mode WSL

Tekan:

```
Ctrl + Shift + P
```

Ketik:

```
WSL: New Window
```

Pilih:
Open Folder → `/home/username/bigdata-project`

Di kiri bawah akan muncul:
`WSL: Ubuntu`

Artinya VS Code sekarang berjalan di Linux environment.

STEP 9 — Install Extension Python

Install:

`Python (Microsoft)`

STEP 10 — Setup Virtual Environment

Di terminal VS Code:

```
python3 -m venv venv
source venv/bin/activate
pip install pyspark
```

STEP 11 — Pilih Interpreter

Tekan:

```
Ctrl + Shift + P
Python: Select Interpreter
```

Pilih:

```
venv/bin/python
```

🧩 BAGIAN E — LETAKKAN DATASET

Step 12 — Masuk ke folder project di WSL

Buka Ubuntu, lalu:

```
cd ~/bigdata-project/data/raw
```

Pastikan sudah di folder itu:

```
pwd
```

Harus muncul:

```
/home/username/bigdata-project/data/raw
```

Step 13 — Buka folder itu di Windows Explorer

Masih di terminal Ubuntu, ketik:

```
explorer.exe .
```

Windows Explorer akan langsung terbuka di folder tersebut.

Step 14 — Copy–Paste File

Sekarang tinggal:

- Buka folder tempat ecommerce_raw.csv ada di Windows
- Copy file
- Paste ke folder Explorer yang baru terbuka

Simpan file:

```
ecommerce_raw.csv
```

Ke:

```
data/raw/ecommerce_raw.csv
```

Minimal 120.000 baris.

🧩 BAGIAN F — IMPLEMENTASI PIPELINE ENTERPRISE

Buat file:

`scripts/batch_pipeline_enterprise.py`

🔥 CODING LENGKAP ENTERPRISE PIPELINE

```
# ============================================================
# ENTERPRISE BATCH DATA PIPELINE
# With Clear Console Output for Classroom
# ============================================================

from pyspark.sql import SparkSession
from pyspark.sql.functions import col, expr, sum as _sum, avg, desc
from pyspark.sql.types import *
import logging
import os
import time

# ========================
# START TIMER
# ========================
start_time = time.time()

print("========================================")
print("   ENTERPRISE BATCH PIPELINE STARTED")
print("========================================")

# ========================
# SETUP LOGGING
# ========================
if not os.path.exists("logs"):
    os.makedirs("logs")

logging.basicConfig(
    filename="logs/batch_pipeline.log",
    level=logging.INFO,
    format="%(asctime)s - %(levelname)s - %(message)s"
)

logging.info("Pipeline started")

# ========================
# INITIALIZE SPARK
# ========================
spark = SparkSession.builder \
    .appName("EnterpriseBatchPipelineDemo") \
    .master("local[*]") \
    .getOrCreate()

spark.sparkContext.setLogLevel("ERROR")
```

```
print("Spark Session Initialized")
print("Spark Version:", spark.version)
print("----------------------------------------")

# ========================
# DEFINE SCHEMA
# ========================
schema = StructType([
    StructField("transaction_id", StringType(), False),
    StructField("customer_id", StringType(), False),
    StructField("product", StringType(), True),
    StructField("category", StringType(), True),
    StructField("price", DoubleType(), True),
    StructField("quantity", IntegerType(), True),
    StructField("transaction_date", StringType(), True)
])

# ========================
# LOAD RAW DATA
# ========================
print("Loading raw dataset...")

df = spark.read \
    .schema(schema) \
    .option("header", True) \
    .csv("data/raw/ecommerce_raw.csv")

df.cache()

raw_count = df.count()
print(f"Total Raw Records      : {raw_count}")
print("----------------------------------------")

# ========================
# CLEANING PHASE
# ========================
print("Cleaning data...")

df_clean = df.dropDuplicates()

df_clean = df_clean.dropna(
    subset=["transaction_id", "customer_id", "price", "quantity"]
)

df_clean = df_clean.filter(
    (col("price") > 0) & (col("quantity") > 0)
)

df_clean = df_clean.withColumn(
    "transaction_date",
    expr("try_to_date(transaction_date, 'yyyy-MM-dd')")
)
```

```
df_clean = df_clean.dropna(subset=["transaction_date"])

clean_count = df_clean.count()

print(f"Total Cleaned Records  : {clean_count}")
print(f"Data Removed           : {raw_count - clean_count}")
print("----------------------------------------")

# =========================
# TRANSFORMATION
# =========================
print("Applying transformation...")

df_transformed = df_clean.withColumn(
    "total_amount",
    col("price") * col("quantity")
)

print("Transformation completed")
print("----------------------------------------")

# =========================
# AGGREGATION
# =========================
print("Calculating business metrics...")

df_curated = df_transformed.groupBy("category") \
    .agg(_sum("total_amount").alias("total_revenue"))

df_top_products = df_transformed.groupBy("product") \
    .agg(_sum("quantity").alias("total_quantity")) \
    .orderBy(desc("total_quantity")) \
    .limit(5)

df_avg_transaction = df_transformed.groupBy("customer_id") \
    .agg(avg("total_amount").alias("avg_transaction_value"))

print("Aggregation completed")
print("----------------------------------------")

# =========================
# DISPLAY RESULTS
# =========================
print("Top 5 Products:")
df_top_products.show()

print("Category Revenue:")
df_curated.show()

print("----------------------------------------")
```

```
# ========================
# SAVE DATA
# ========================
print("Saving Clean Layer (Parquet)...")

df_transformed.write.mode("overwrite") \
    .parquet("data/clean/parquet/")

print("Saving Curated Layer...")

df_curated.write.mode("overwrite") \
    .parquet("data/curated/category_revenue/")

df_top_products.write.mode("overwrite") \
    .parquet("data/curated/top_products/")

df_avg_transaction.write.mode("overwrite") \
    .parquet("data/curated/avg_transaction/")

print("Saving Partitioned Data...")

df_transformed.write.mode("overwrite") \
    .partitionBy("category") \
    .parquet("data/clean/partitioned_by_category/")

print("----------------------------------------")
print("All data successfully saved.")
print("----------------------------------------")

# ========================
# STOP SPARK
# ========================

spark.stop()

end_time = time.time()
execution_time = round(end_time - start_time, 2)

print("========================================")
print(" PIPELINE COMPLETED SUCCESSFULLY ")
print(f" Total Execution Time: {execution_time} seconds")
print("========================================")

logging.info("Pipeline completed successfully")
```

🧩 BAGIAN G — MENJALANKAN PIPELINE

Aktifkan venv:

```
source venv/bin/activate
```

Jalankan:

```
python scripts/batch_pipeline_enterprise.py
```

📊 BANDINGKAN UKURAN CSV VS PARQUET

```
du -sh data/clean/parquet/
```

Parquet jauh lebih efisien.

🎓 KONSEP YANG DIKUASAI

Dengan modul ini kita memahami:

- Linux server environment
- Remote development via VS Code WSL
- Virtual environment
- Explicit schema (production best practice)
- Lazy evaluation Spark
- Action vs Transformation
- Medallion architecture
- Aggregation analytics
- Columnar storage (Parquet)
- Partitioning strategy
- Storage optimization
- Enterprise data pipeline mindset

1️⃣ Linux Server Environment

Lingkungan kerja berbasis Linux yang menyerupai server production.
Di industri:

- Spark cluster berjalan di Linux
- Deployment dilakukan di server Linux
- Data engineer bekerja via terminal (CLI)

Menggunakan Linux melatih:

- Command line skill
- Struktur file system server
- Environment configuration

👉 Ini mendekatkan kita ke kondisi real production.

2 Remote Development via VS Code WSL

VS Code WSL memungkinkan coding di Windows tetapi eksekusi berjalan di Linux (WSL).

Konsepnya:

- Editor: berjalan di Windows
- Runtime & file system: berjalan di Linux

Keuntungannya:

- Developer experience nyaman
- Lingkungan tetap server-grade
- Tidak perlu dual boot

👉 Ini praktik modern dalam development hybrid environment.

---

3 Virtual Environment

Virtual environment (venv) adalah lingkungan Python terisolasi untuk satu project.

Tujuannya:

- Menghindari konflik versi library
- Menjaga dependency tetap konsisten
- Membuat project reproducible

Di industri:
Setiap project selalu memiliki environment terpisah.

👉 Ini bagian dari dependency management best practice.

---

4 Explicit Schema (Production Best Practice)

Schema didefinisikan secara manual sebelum membaca data.
Mengapa penting?

- Lebih cepat daripada inferSchema
- Menghindari kesalahan tipe data
- Konsisten dengan data warehouse schema

Production system tidak mengandalkan auto-detection.

👉 Ini memastikan data governance dan reliability.

---

5 Lazy Evaluation Spark

Spark tidak langsung mengeksekusi perintah.
Ia:

- Membangun logical plan (DAG)
- Mengeksekusi hanya saat diperlukan

Eksekusi terjadi saat ada Action seperti:

- `count()`
- `show()`
- `write()`

Keuntungannya:

- Optimasi otomatis
- Eksekusi lebih efisien

👉 Ini inti performa Spark.

---

6 Action vs Transformation

Transformation:

- `filter()`
- `select()`
- `groupBy()`
- `withColumn()`

Tidak langsung dieksekusi.

Action:

- `count()`
- `show()`
- `collect()`
- `write()`

Memicu eksekusi job.

👉 Memahami ini penting untuk mengontrol performa dan biaya komputasi.

---

7 Medallion Architecture

Arsitektur data lake berlapis:

- Raw (Bronze) → data mentah
- Clean (Silver) → data tervalidasi
- Curated (Gold) → siap analitik

Keuntungannya:

- Data tidak pernah ditimpa
- Jejak perubahan jelas
- Mendukung audit & governance

👉 Standar modern dalam data engineering.

---

8 Aggregation Analytics

Proses menghitung metrik bisnis dari data mentah.
Contoh:

- *Total revenue*
- *Top selling product*
- *Average transaction*

Teknik yang digunakan:

- `groupBy()`
- `sum()`
- `avg()`

👉 Ini menjembatani ***data engineering*** dan ***business intelligence***.

---

9 Columnar Storage (Parquet)

Parquet adalah format penyimpanan berbasis kolom.

Keunggulan:

- Lebih kecil ukuran file
- Lebih cepat untuk query analitik
- Menyimpan metadata schema

Berbeda dengan CSV (row-based).

👉 Standar industri untuk data lake dan big data.

---

10 Partitioning Strategy

Membagi data ke folder berdasarkan kolom tertentu.

Contoh:

```
category=Electronics/
category=Fashion/
```

Keuntungannya:

- Query lebih cepat
- Spark hanya membaca partisi relevan
- Mengurangi IO

Disebut juga: Partition Pruning.

👉 Penting untuk data skala besar.

---

11 Storage Optimization

Strategi untuk:

- Mengurangi ukuran file
- Mengurangi waktu baca
- Menghemat biaya storage

Teknik:

- Gunakan Parquet
- Gunakan compression
- Gunakan partitioning
- Hindari file kecil berlebihan

👉 Efisiensi storage = efisiensi biaya cloud.

1️⃣2️⃣ Enterprise Data Pipeline Mindset

Cara berpikir seperti data engineer profesional:

- Tidak overwrite data mentah
- Selalu validasi kualitas data
- Gunakan schema eksplisit
- Logging setiap proses
- Gunakan format efisien
- Optimalkan performa

Fokusnya bukan sekadar:
`"Script berjalan"`

Tetapi:
`"Pipeline scalable, maintainable, dan production-ready"`

🏁 STATUS AKHIR

Jika:
```
python scripts/batch_pipeline_enterprise.py
```

Berjalan tanpa error dan folder:

```
data/clean/
data/curated/
```

Terbentuk dengan Parquet dan partitioning,

Maka:

✅ Praktikum 2 dinyatakan **SUKSES (Enterprise Level)**

Praktikum 2 ini memang dominan di **Processing Layer**
tapi sebenarnya sudah menyentuh beberapa layer lain.

🏗 Arsitektur Big Data Secara Umum

Secara sederhana biasanya dibagi menjadi:

1. Data Source Layer
2. Ingestion Layer

3. Storage Layer
4. Processing Layer
5. Serving / Analytics Layer
6. Visualization / BI Layer

🔍 **Posisi Praktikum 2**

1️⃣ **Data Source Layer**

Dalam praktikum:

`ecommerce_raw.csv`

Itu adalah simulasi data dari:

- E-commerce system
- Database transaksi
- ERP

👉 Jadi layer ini sudah ada (walau masih file statis).

2️⃣ **Ingestion Layer**

Saat kita melakukan:

`spark.read.csv(...)`

Itu adalah **batch ingestion**.
Namun:

- Belum streaming
- Belum real-time
- Belum dari API / Kafka

👉 Jadi ini masih ingestion sederhana (batch mode).

3️⃣ **Storage Layer**

Ketika kita simpan:

```
.parquet("data/clean/")
Dan pakai partitioning:
.partitionBy("category")
```

Itu sudah masuk ke:

👉 **Data Lake Storage Layer**

Walau masih lokal, konsepnya sama seperti:

- *Amazon S3*
- *Azure Data Lake*
- *Google Cloud Storage*

---

4 Processing Layer (Bagian Terbesar)

Ini inti praktikum 2, yang kita lakukan:

- *Cleaning*
- *Filtering*
- *Validation*
- *Transformation*
- *Aggregation*
- *Ranking*
- *Partitioning*

Itu semua adalah:
👉 **Batch Processing Layer menggunakan Apache Spark**

---

5 Serving / Analytics Layer

Kita sudah sedikit menyentuh ini ketika:

```
df_curated = groupBy().agg()
```

Itu sebenarnya sudah membangun dataset siap BI (*Business Intelligence*).
Tapi:

- Belum expose ke dashboard
- Belum ke data warehouse
- Belum ke API

Jadi layer ini masih minimal.

🎯 Praktikum 2 ini:

Layer	Status
Source	✓ Simulasi
Ingestion	✓ Batch
Storage	✓ Data Lake (Parquet)
Processing	✅ FOKUS UTAMA
Serving	⚠️ Minimal
Visualization	❌ Belum

🧠 Level Arsitektur Kita Sekarang

Secara kematangan arsitektur:

- Sudah di level **Data Engineering Foundation**
- Belum masuk ke:
  - *Real-time streaming*
  - *Orchestration*
  - *Data warehouse integration*
  - *BI layer*

📋 1️⃣ CHECKLIST VALIDASI PRAKTIKUM 2

Batch Data Ingestion & Processing (Enterprise Edition)

Checklist berikut untuk memastikan pipeline benar-benar berjalan.

✅ A. Environment & Setup

No	Item Validasi	Status
1	WSL Ubuntu berhasil terinstall	☐
2	VS Code terhubung ke WSL	☐
3	Virtual environment aktif	☐
4	PySpark berhasil diinstall	☐
5	Spark Session berhasil dijalankan	☐

✅ B. Struktur Project

No	Item Validasi	Status
6	Struktur folder raw/clean/curated ada	☐
7	Dataset ditempatkan di data/raw/	☐
8	Folder logs terbentuk	☐

✅ C. Processing & Transformation

No	Item Validasi	Status
9	Schema eksplisit digunakan (tidak inferSchema)	☐
10	Data cleaning dilakukan (dropDuplicates, dropna, filter)	☐
11	Kolom total_amount dibuat	☐
12	Aggregasi per category berhasil	☐
13	Top 5 produk dihitung	☐
14	Rata-rata transaksi per customer dihitung	☐

✅ D. Storage & Optimization

No	Item Validasi	Status
15	Data disimpan dalam format Parquet	☐
16	Partitioning berhasil dibuat	☐
17	Folder partisi terbentuk (misal category=...)	☐
18	Ukuran Parquet lebih kecil dari CSV (jika diuji)	☐

---

2 OUTPUT WAJIB YANG DIKUMPULKAN

Wajib mengumpulkan:

A. Source Code

- File: batch_pipeline_enterprise.py
- Harus dapat dijalankan tanpa error

B. Screenshot / Bukti Eksekusi

Minimal menampilkan:

1. Spark Version
2. Total Raw Records
3. Total Cleaned Records
4. Top 5 Products (df.show())
5. Category Revenue
6. Pesan: PIPELINE COMPLETED SUCCESSFULLY

C. Struktur Folder Output

Wajib menunjukkan folder:

data/clean/parquet/
data/clean/partitioned_by_category/
data/curated/top_products/
data/curated/category_revenue/

D. Analisis Singkat (1–2 Halaman PDF)

Berisi:

1. Penjelasan singkat Batch Processing
2. Mengapa Parquet lebih efisien
3. Penjelasan Partition Pruning
4. Hasil Top 5 produk
5. Insight bisnis sederhana dari hasil Praktikum ini.

---

3 RUBRIK PENILAIAN (SKOR 1–4)

Skala:

Skor	Kriteria Umum
4	Sangat Baik (Enterprise-ready)
3	Baik
2	Cukup
1	Kurang

A. Setup & Environment (20%)

Skor	Deskripsi
4	Environment WSL & venv berjalan sempurna
3	Ada minor issue tapi pipeline jalan
2	Banyak error setup
1	Tidak bisa menjalankan Spark

B. Implementasi Pipeline (30%)

Skor	Deskripsi
4	Schema eksplisit, cleaning lengkap, agregasi benar
3	Pipeline berjalan tapi ada bagian kurang optimal
2	Hanya load & save tanpa transformasi lengkap
1	Script tidak lengkap / error

C. Storage & Optimization (20%)

Skor	Deskripsi
4	Parquet + Partitioning benar dan dapat dijelaskan
3	Parquet benar tapi tanpa partition optimal
2	Hanya CSV
1	Tidak ada penyimpanan benar

D. Analisis & Pemahaman Konsep (20%)

Skor	Deskripsi
4	Dapat menjelaskan konsep batch, distributed, parquet
3	Mengerti sebagian konsep
2	Penjelasan sangat dangkal
1	Tidak memahami konsep

E. Kerapian & Profesionalisme (10%)

Skor	Deskripsi
4	Struktur rapi, logging ada, output jelas
3	Rapi tapi minim dokumentasi
2	Kurang rapi
1	Tidak terstruktur

🎯 **Skor Akhir**

Nilai akhir =

(Setup×20%) +
(Implementasi×30%) +
(Storage×20%) +
(Konsep×20%) +
(Profesionalisme×10%)

🎓 **Insight**

Praktikum 2 ini bukan cuma "mengolah CSV".

Secara arsitektur sudah belajar:

- *Data Lake*
- *Distributed Processing*
- *Batch Pipeline*
- *Partitioning Strategy*
- *Storage Optimization*

Itu sudah fondasi kuat untuk ***Data Engineer***.